Diseño de dos metaheurísticas para configurar la extracción de métricas en fichas de modelos de Hugging Face

Juan Felipe Gallo
Maestría en Ingeniería, Universidad de Antioquia, Medellín, Colombia
Metaheurística 2026-01, Entrega 2

***Resumen*—Se describe el diseño de dos metaheurísticas para configurar la etapa de extracción de métricas de un *pipeline* ETL sobre fichas de modelos: un GRASP con búsqueda local y memoria de evaluaciones, y un algoritmo genético. Ambos comparten la representación de la solución (reglas ordenadas y un umbral), la función de evaluación y el vecindario de movimientos, de modo que difieren solo en la estrategia de búsqueda y puedan compararse con el mismo presupuesto de evaluaciones.**

***Index Terms*—GRASP, algoritmo genético, optimización combinatoria, extracción de información.**

I. Problema, solución y evaluación

I-A. Representación

Se dispone de un catálogo de m reglas de extracción; cada una combina un analizador (línea `nombre: valor`, fila de tabla o columna de tabla) con una familia de métricas. Una solución es $x = (\pi, \theta)$, donde π es una tupla *ordenada* de a lo sumo K reglas distintas y $\theta \in \Theta = \{1, 100, 1000\}$ es una cota superior de plausibilidad para los valores aceptados. El orden importa: para cada ficha y métrica, la primera regla de π que produce un valor gana y las siguientes se descartan para esa métrica.

I-B. Evaluación

La salida de cada regla sobre cada ficha se calcula una sola vez y se guarda; evaluar una solución consiste en combinar esas salidas según π y θ. Sobre las fichas con referencia (el bloque `model-index`, oculto al extractor) se obtienen precisión y cobertura por etiqueta de tarea $\tau \in T$. La aptitud es el F_1 macro, $\bar{F}_1(x) = \frac{1}{|T|}\sum_\tau F_{1,\tau}(x)$, sujeto a precisión global $P(x) \geq 0{,}90$ y tiempo por ficha $t(x) \leq t_{\max}$. Las restricciones son duras: con $v(x) = \max(0, 0{,}90 - P) + \max(0, t/t_{\max} - 1)$,

$$f(x) = \begin{cases} \bar{F}_1(x) & v(x) = 0, \\ -1 - v(x) & v(x) > 0. \end{cases} \tag{1}$$

Toda solución infactible vale menos que cualquier factible, y v conduce la búsqueda hacia la región factible.

I-C. Vecindario

Ambos algoritmos usan los mismos cinco movimientos: *insertar* una regla en una posición, *eliminar* una, *reemplazar* una por otra no usada, *intercambiar* dos posiciones y *cambiar* θ. Con $K = 5$ y $m = 27$ el vecindario tiene unos 260 elementos.

Algorithm 1 GRASP con memoria

Require: presupuesto B, parámetro α, K
$x^* \leftarrow \emptyset$
while evaluaciones reales $< B$ **do**
  $\pi \leftarrow ()$, $\theta \leftarrow$ aleatorio en Θ
  while $|\pi| < K$ **do**
    evaluar todas las inserciones (regla, posición)
    $RCL \leftarrow \{c : f(c) \geq f_{\max} - \alpha(f_{\max} - f_{\min})\}$
    si ninguna inserción mejora $f(x)$, terminar; si no, elegir $c \in RCL$ al azar y actualizar π
  end while
  $x \leftarrow$ búsqueda local de mejor mejora desde x
  if $f(x) > f(x^*)$ **then**
    $x^* \leftarrow x$
  end if
end while
return x^*

II. Metaheurística con memoria y aleatoriedad: GRASP

GRASP [1] alterna una construcción voraz aleatorizada con una búsqueda local (Algoritmo 1).

Construcción. Se parte de una tupla vacía y un θ al azar. En cada paso se evalúan todas las inserciones posibles de reglas no usadas en cualquier posición, y se elige al azar dentro de la lista restringida de candidatos (RCL) definida por α: con $\alpha = 0$ es un voraz puro y con $\alpha = 1$ es aleatorio. Termina al llegar a K reglas o cuando ninguna inserción mejora la aptitud.

Exploración. La búsqueda local usa *mejor mejora*: recorre todo el vecindario y se mueve al mejor vecino, hasta un óptimo local. Se elige esta estrategia porque el vecindario es pequeño y sus evaluaciones se reutilizan gracias a la memoria.

Memoria. Un diccionario guarda cada configuración evaluada con su aptitud. Los reinicios y la búsqueda local no reevalúan lo ya visto, y el presupuesto cuenta solo evaluaciones reales.

Criterio de parada. Se agota el presupuesto B de evaluaciones reales. Se devuelve la mejor solución hallada.

Parámetros y valores iniciales. $\alpha \in [0, 1]$, con valor inicial 0,3 y rango a explorar $\{0; 0{,}1; 0{,}3; 0{,}5; 0{,}8\}$; B con valor inicial 3.000 y rango $\{500; 1500; 3000\}$. K, $t_{\max}$ y la precisión mínima son datos del problema.

Algorithm 2 Algoritmo genético

Require: N, p_c, p_m, e, presupuesto B
inicializar población de N soluciones; evaluarla
while evaluaciones reales $< B$ **y** no estancado **do**
  $\mathcal{N} \leftarrow$ las e mejores (elitismo)
  while $|\mathcal{N}| < N$ **do**
    elegir padres a, b por torneo de tamaño k
    con prob. p_c: $h \leftarrow \text{cruce}(a, b)$; si no, $h \leftarrow a$
    con prob. p_m: aplicar un movimiento aleatorio a h
    agregar h a $\mathcal{N}$ si no es duplicado; evaluar
  end while
  población $\leftarrow \mathcal{N}$
end while
return mejor solución

III. Metaheurística poblacional: algoritmo genético

El algoritmo genético [2] mantiene N soluciones y las mejora mediante selección, cruce y mutación (Algoritmo 2).

Representación. La misma $x = (\pi, \theta)$: lista ordenada de a lo sumo K reglas distintas y un umbral.

Población inicial. Una fracción ρ de los individuos se construye con la construcción aleatorizada de GRASP ($\alpha = 0{,}3$) y el resto se genera al azar, con un tamaño $k \in \{1, \ldots, K\}$, reglas distintas al azar y θ aleatorio. La mezcla aporta calidad inicial sin perder diversidad.

Selección. Torneo de tamaño k_t sobre la aptitud f; las infactibles quedan por debajo de las factibles.

Cruce. Cruce por orden: se elige un punto de corte en el padre a, el hijo conserva ese prefijo y completa con las reglas del padre b, en el orden de b, que aún no aparecen, hasta K reglas o agotar las de b. El umbral θ se hereda de uno de los padres con igual probabilidad. El cruce mantiene reglas distintas y respeta la importancia del orden.

Mutación. Con probabilidad p_m se aplica un movimiento aleatorio del vecindario (insertar, eliminar, reemplazar, intercambiar o cambiar θ), lo que mantiene la coherencia con GRASP.

Reemplazo. Generacional con elitismo: pasan las e mejores y el resto de la nueva población proviene de los hijos. No se admiten individuos duplicados; si un hijo ya existe, se muta de nuevo.

Memoria. Se usa la misma caché de evaluaciones que en GRASP.

Criterio de parada. Se agota el presupuesto B de evaluaciones reales, igual al de GRASP, o la mejor aptitud no mejora en G generaciones consecutivas.

Parámetros y valores iniciales. $N = 50$ (rango $\{30, 50, 100\}$); $p_c = 0{,}9$ (rango $\{0{,}7; 0{,}9\}$); $p_m = 0{,}2$ (rango $\{0{,}1; 0{,}2; 0{,}4\}$); $k_t = 3$; $e = 2$; $\rho = 0{,}2$; $G = 15$; $B = 3000$.

IV. Comparación prevista

Los dos algoritmos se compararán con el mismo presupuesto de evaluaciones, las mismas instancias y réplicas con semillas distintas, además de una línea base voraz y una búsqueda aleatoria. Sus resultados se contrastarán con una prueba estadística pareada sobre las réplicas, y en instancias pequeñas con la enumeración exacta como referencia de optimalidad.

Referencias

[1] T. A. Feo and M. G. C. Resende, "Greedy randomized adaptive search procedures," *J. Global Optim.*, vol. 6, no. 2, pp. 109–133, 1995.

[2] M. Gendreau and J.-Y. Potvin, Eds., *Handbook of Metaheuristics*, 3rd ed. Cham, Switzerland: Springer, 2019.